

ПОСОБИЕ ПО РАБОТЕ С JETVOT_MIREA

ЧАСТЬ 2. ДРАЙВЕРА ДЛЯ РАБОТЫ С «ЖЕЛЕЗОМ»

Разработано в РТУ МИРЭА ИИИ КПУ
Грачевым А.В.
Зениной А.А.

ОГЛАВЛЕНИЕ

Подключение к Jetson Nano	4
Управление режимом питания Jetson nano	5
Python3-библиотека «jetbot»	6
1. Краткое описание	6
2. Возможности	6
3. Ссылки	6
4. Развёртывание	6
Реализация приводного уровня в ПО (прошивке) ESP32 - «esp_firmware»	6
1. Краткое описание	6
2. Возможности	7
3. Использование	7
4. Полезные ссылки	8
ROS2-пакет для взаимодействия с ESP32 через SerialPort - «serial_bridge_package»	8
1. Краткое описание	8
2. Описание	8
2.1. Нода twist_to_command	9
2.2. Нода serial_bridge_node	9
2.3. Нода feedback_processor	10
2.4. Launch-файл	11
3. Особенности	14
4. Развёртывание	14
ROS2-пакет для настройки ПИД-регуляторов для моторов ESP32 - «simple_regulators_configurator_pkg»	14
1. Краткое описание	14
2. Возможности	14
3. Использование	15
3.1. Проводная настройка	15
3.2. Беспроводная настройка	15
3.2.1. Jetson Nano	16
3.2.2. Ваш компьютер (хост)	16
Python3-приложение для настройки ПИД-регуляторов для моторов ESP32 - «simple_PID_configurator»	16
1. Краткое описание	16
2. Возможности	16
3. Использование	17
ROS2-пакет для взаимодействия с лидаром - «slidar_ros2»	17
1. Краткое описание	17
2. Возможности	18
3. Использование	18
4. Ссылки	18
ROS2-пакет для взаимодействия с камерой глубины	18
Оптическая камера	18
Mini-RX MarvelMind	19
Полезные ссылки	19
Обратная связь	19
ПРИЛОЖЕНИЕ А	20
1. Установка Arduino-IDE	20
2. Установка дополнительных библиотек	23

3. Прошивка ESP32	26
4. Проверка работы ПО	31
ПРИЛОЖЕНИЕ Б	33
Сообщения-команды (входящие)	33
1. Управление скоростями платформы.	33
2. Управление скоростями каждого из колёс.	33
3. Установка коэффициентов ПИД-регуляторов для управления скоростью моторов.	33
4. Запись коэффициентов ПИД-регуляторов для управления скоростью моторов во Flash-память (энергонезависимую).	34
5. Запрос текущих коэффициентов ПИД-регуляторов.	34
Сообщения-ответы (исходящие)	34
1. Получение одометрии и скоростей платформы и колёс.	34
2. Получение текущих коэффициентов ПИД-регуляторов для управления скоростью моторов. (ответ на сообщение-команду 5)	35
ПРИЛОЖЕНИЕ В	37
Немного о том, что происходит под капотом этого скрипта	37
ПРИЛОЖЕНИЕ Г	38

Подключение к Jetson Nano

Доступ к вычислителю (Jetson Nano) возможно получить 2 способами:

- **непосредственно** - всю периферию (монитор, мышь, клавиатура) подключаем к Jetson Nano напрямую (втыкаем в порты на плате);
- **удалённо, через SSH**. Подробнее про SSH можно прочитать здесь <https://habr.com/ru/articles/802179/>.

Непосредственное подключение нужно, чтобы попасть на рабочий стол операционной системы, которая установлена на Jetson (специальная Ubuntu 20.04). Там можно произвести подключение к Wi-Fi и другие настройки. Так же это удобно для отладки - посмотреть на логгер загрузки ОС, там показаны критические ошибки (проблемы ядра, критического аппаратного обеспечения или графики).

Для работы с роботом с помощью другого (своего) компьютера, можно подключиться удалённо используя протокол SSH. Условием удалённого подключения является нахождение сервера (машины, к которой происходит подключение) и хоста (машины, с которой происходит подключение) в одной сети Wi-Fi.

Для подключения через SSH к любому компьютеру (серверу) нужно знать такие параметры как:

- IP-адрес сервера в общей с нами сети.
- Имя пользователя в ОС на сервере.
- Пароль от учётной записи пользователя на сервере.

Шаблон команды SSH-подключения представлен ниже.

```
ssh user@192.168.0.1
```

(192.168.0.1 - какой-то пример IP-адреса)

После выполнения команды необходимо ввести пароль от записи пользователя на сервере.

Для нашего случая команда подключения имеет вид:

```
ssh jetson<ip-address>
```

где jetson - имя пользователя в ОС на сервере, а <ip-address> необходимо заменить на IP-адрес вашего робота, показывающийся на OLED-дисплее. Также IP-адрес можно узнать из настроек сети при непосредственном подключении (через терминал команда «ip a»).

После выполнения команды, вводим пароль:

```
jetbot
```

После удачного подключения строка-приглашения должна изменить свой вид.

Управление режимом питания Jetson nano

На Jetson Nano есть два основных режима, которые отличаются количеством активных ядер и частотой :

- Режим MAXN (ID: 0) — это режим максимальной производительности (10 Вт). Используются все 4 ядра процессора на максимальной частоте.
- Режим 5W (ID: 1) — это энергосберегающий режим (5 Вт). Активны только 2 ядра (CORE_0 и CORE_1), что снижает энергопотребление и нагрев.

Посмотреть текущий режим

Откройте терминал и выполните команду:

```
sudo nvpmode -q
```

В ответ вы увидите строку NV Power Mode: MAXN (или 5W), а также цифру (0 или 1), соответствующую режиму .

Переключить режим

Используйте команду:

```
sudo nvpmode -m <номер_режима>
```

- Для режима 4 ядер (MAXN/10W): `sudo nvpmode -m 0`
- Для режима 2 ядер (5W): `sudo nvpmode -m 1`

Настройка сохраняется после перезагрузки.

Дополнительно

Для принудительной установки максимальных частот (помимо включения ядер) используется команда `sudo jetson_clocks` .

Удобно следить за состоянием ядер и частот с помощью утилиты `jtop` (установка: `sudo pip install jetson-stats`) .

Python3-библиотека «jetbot»

1. Краткое описание

Это python3-библиотека для взаимодействия с исходной версией jetbot (без модификаций и дополнений, произведённых в РТУ МИРЭА).

2. Возможности

Библиотека позволяет программировать простое движение робота (оригинального jetbot), а также обеспечивает нормальную работу его внутренних систем (к примеру, вывод информации на OLED-дисплей). Управлять модифицированным роботом с этой библиотекой не получится, так как мы изменили конструкцию самого робота, но эта библиотека необходима для нормальной работы штатных систем. Также в пакете (библиотеке) есть jupyter-notebooks, в которых описывается взаимодействие с роботом, возможно полезное для разработчика.

3. Ссылки

Python-пакет на Github: <https://github.com/waveshare/jetbot/tree/master>

4. Развёртывание

Подключимся к Jetson Nano (любым способом) и выполним последовательно команды в терминале.

```
git clone https://github.com/waveshare/jetbot.git
cd jetbot
pip3 install .
```

Реализация приводного уровня в ПО (прошивке) ESP32

- «esp_firmware»

1. Краткое описание

«esp_firmware» - это проект для Arduino IDE, в котором находится код программы микроконтроллера ESP32 для реализации приводного уровня.

2. Возможности

Данная программа позволяет роботу использовать обратную связь по скорости колёс для точного управления роботом. В ПО реализованно:

- **Управление роботом осуществляется путём задания желаемых скоростей для моторов командами, использующими единицы СИ (м/с, рад/с).** Доступны два режима управления:
 - Скоростью мобильной платформы: заданием вектора, включающего линейную скорость вдоль продольной оси X и угловую скорость вращения вокруг вертикальной оси Z.
 - Скоростью колёс: отдельным заданием скоростей для левого и правого колёс.
- **ПИД-регуляция каждого мотора по скорости.** Для каждого мотора реализован ПИД-регулятор с возможностью настройки.
- **Алгоритм записи значений коэффициентов регуляторов в Flash-памяти (энергонезависимой).** Запись в энергонезависимую память позволяет сохранять значения даже при выключении устройства и не сбрасываться к значениям «по умолчанию»
- **Подсчёт одометрии робота.** С момента запуска робот рассчитывает своё пространственное положение (x, y, theta (угол поворота)) и отправляет на главный вычислитель (Jetson Nano). Одометрия подсчитывается с постоянно увеличивающейся ошибкой, что совершенно нормально.
- **Аварийная остановка при зависании Jetson Nano.** Если с Jetson Nano сообщения не приходят в течение 5 секунд, то робот останавливается.

Также в коде используются такие технологии как:

- **Обработка прерываний и машина состояний.** Необходимы для точного подсчёта скорости и определения направления вращения в «фоновом» режиме (не занимая активное время программы).
- **Кастомный протокол поверх Serial Port.** Для передачи информации между Jetson Nano и ESP32 был разработан свой протокол связи.

3. Использование

В приложении А показано, что нужно сделать, чтобы залить на ESP32 данную прошивку. Протокол, с помощью которого можно взаимодействовать с ESP32 представлен в приложении Б. В

нормальном режиме ESP32 отправляет на Jetson Nano сообщения-ответы типа 1 (одометрия). При отправке на ESP32 сообщения-команды типа 5 (запрос коэффициентов ПИД-регуляторов) ESP32 отправляет сообщение-ответ типа 2 (коэффициенты текущих ПИД-регуляторов).

4. Полезные ссылки

- Урок от Alex Gyver по ПИД регуляторам: <https://alexgyver.ru/lessons/pid/?ysclid=ml010i8tz9271808957>
- Урок от Alex Gyver для работы с внешними прерываниями: <https://alexgyver.ru/lessons/interrupts/>
Урок от Alex Gyver для работы с SerialPort: <https://alexgyver.ru/lessons/uart/>
- Урок от Alex Gyver по программным таймерам: <https://alexgyver.ru/lessons/soft-timer/>
- Урок от Alex Gyver по инкрементальным энкодерам: <https://alexgyver.ru/lessons/encoder/>
- Урок от Alex Gyver по ESP32: <https://alexgyver.ru/lessons/about-esp/>
- Статья на Habr про обработку значений с инкрементального энкодера с использованием машины состояний: <https://habr.com/ru/articles/729382/?ysclid=ml01c6z26233038659>
- Статья про использование Flash-памяти в ESP32: <https://microcontrollerslab.com/save-data-esp32-flash-permanently-preferences-library/>

ROS2-пакет для взаимодействия с ESP32 через

SerialPort - «serial_bridge_package»

1. Краткое описание

Это ROS2-пакет, обеспечивающий корректную работу с Serial Port и протоколом, по которому работает ESP32.

2. Описание

В пакете содержатся 3 ноды:

- *twist_to_command* - преобразователь из сообщений-команд типа ROS2 geometry_msgs/msg/Twist в протокольное сообщение для ESP32

- *serial_bridge_node* - мост между ROS2 и UART (Serial Port)
- *feedback_processor* - преобразователь из протокольных сообщений ESP32 в сообщения типа ROS2 (несколько типов)

Рассмотрим каждую из них более подробно.

2.1. Hoda twist to command

Нода является конвертером сообщений Twist в кастомный протокол для ESP32 с форматом \$linear;angular#. Нода подписывается на стандартный топик /cmd_vel и извлекает линейную скорость по X и угловую по Z, игнорируя остальные компоненты. А после публикует в топик /esp32_input строковые сообщения протокольного формата.

В таблице 1 представлены используемые интерфейсы в системе ROS2.

Таблица 1. Используемые интерфейсы ROS2 в ноде.

Тип взаимодействия	Название	Тип сообщения	Комментарий
Подписка	/cmd_vel	geometry_msgs/msg/Twist	Вектор управления
Публикация	/esp32_input	std_msgs/msg/String	Протокольное сообщение

Параметры в ноде отсутствуют.

Запуск осуществляется командой

```
ros2 run serial_bridge_package twist_to_command
```

2.2. Hoda serial bridge node

ROS2-мост для двунаправленной связи между UART-устройствами и ROS-топиками с буферизацией и парсингом сообщений в формате \$message#. Читает сообщения из топика esp32_input (можно настроить) и по сериал порту отправляет их на ESP32. Использует многопоточность для асинхронного чтения порта, поддерживает настраиваемые параметры подключения. Так же публикует сообщения получаемые от ESP32 в отдельный топик (можно настроить).

Требует прав доступа к последовательному порту и стабильного соединения с устройством. В таблице 2 представлены используемые интерфейсы в системе ROS2. Значения указанные в <> являются параметрами (см. далее).

Таблица 2. Используемые интерфейсы ROS2 в ноде.

Тип взаимодействия	Название	Тип сообщения	Комментарий
Подписка	/<ros_in_topic>	std_msgs/msg/String	Сообщение-команда, отправляемая на ESP32
Публикация	/<ros_out_topic>	std_msgs/msg/String	Сообщение-ответ, получаемый от ESP32

В таблице 3 представлены параметры ноды.

Таблица 3. Параметры ноды.

Имя параметра	Тип	Значение по умолчанию	Комментарий
serial_port	string	dev/esp32	Точка монтирования подключенного устройства
baudrate	int	115200	Скорость работы Serial Port
ros_out_topic	string	serial/from	Имя ROS2-топика, в котором публикуются сообщения, полученные от ESP32 через Serial Port
ros_in_topic	string	serial/to	Имя ROS2-топика, из которого сообщения пересылаются на ESP32 через Serial Port

Запуск осуществляется командой

```
ros2 run serial_bridge_package serial_bridge_node
```

2.3. Нода feedback processor

ROS2-узел, который читает топик сообщений от ESP32 и парсит их, преобразует из строк в числа и публикует далее в отдельные топики. Нода обрабатывает сырые данные с ESP32 в формате \$data1;...;dataN# и публикует в темы одометрии, скоростей колёс и ПИД-коэффициентов. Преобразует углы в кватернионы и публикует преобразования систем координат через TF (<frame_id> => <child_frame_id>). Ожидает строго форматированные сообщения с N числовыми полями без обработки ошибок связи. В таблице 4 представлены используемые интерфейсы в системе ROS2.

Таблица 4. Используемые интерфейсы ROS2 в ноды.

Тип взаимодействия	Название	Тип сообщения	Комментарий
Подписка	/esp32_feedback	std_msgs/msg/String	Сообщение-ответ, получаемый от ESP32
Публикация	/odom	nav_msgs/msg/Odometry	Одометрия
Публикация	/PIDcoeffs	std_msgs/msg/String	Текущие коэффициенты ПИД-регуляторов для управления моторами
Публикация	/sensors/wheel/right/speed	std_msgs/msg/Float32	Скорость правого колеса в м/с
Публикация	/sensors/wheel/left/speed	std_msgs/msg/Float32	Скорость левого колеса в м/с

В таблице 5 представлены параметры ноды.

Таблица 5. Параметры ноды.

Имя параметра	Тип	Значение по умолчанию	Комментарий
frame_id	string	odom	Имя фрейма одометрии
child_frame_id	string	base_footprint	Имя фрейма для связи с существующим деревом-преобразований модели робота

Запуск осуществляется командой

```
ros2 run serial_bridge_package feedback_processor
```

2.4. Launch-файл

Для простоты запуска всей системы был создан Launch-файл, запускающий весь набор нод в некоторой конфигурации. Развёрнутая конфигурация представлена в таблице 6. Все ноды имеют имена такие же, как и точки входа из файла установки (setup.py). Также все элементы находятся в пространстве имён «low_level».

Таблица 6. Содержание Launch-файла.

Нода	Параметры	Дополнительные параметры
twist_to_command	-	Перепривязка топиков: «/esp32_input» => «serial/cmd»
serial_bridge_node	serial_port: «/dev/esp32» baudrate: 115200 ros_in_topic: «serial/cmd» ros_out_topic: «serial/feedback»	
feedback_processor	frame_id: «odom» child_frame_id: «base_footprint»	Перепривязка топиков: «/esp32_feedback» => «serial/feedback»

Запуск осуществляется командой

```
ros2 launch serial_bridge_package serial_bringup.launch.py
```

На рисунке 1 представлен RQT-граф, показывающий как взаимодействуют ноды, запускаемые Launch-файлом.

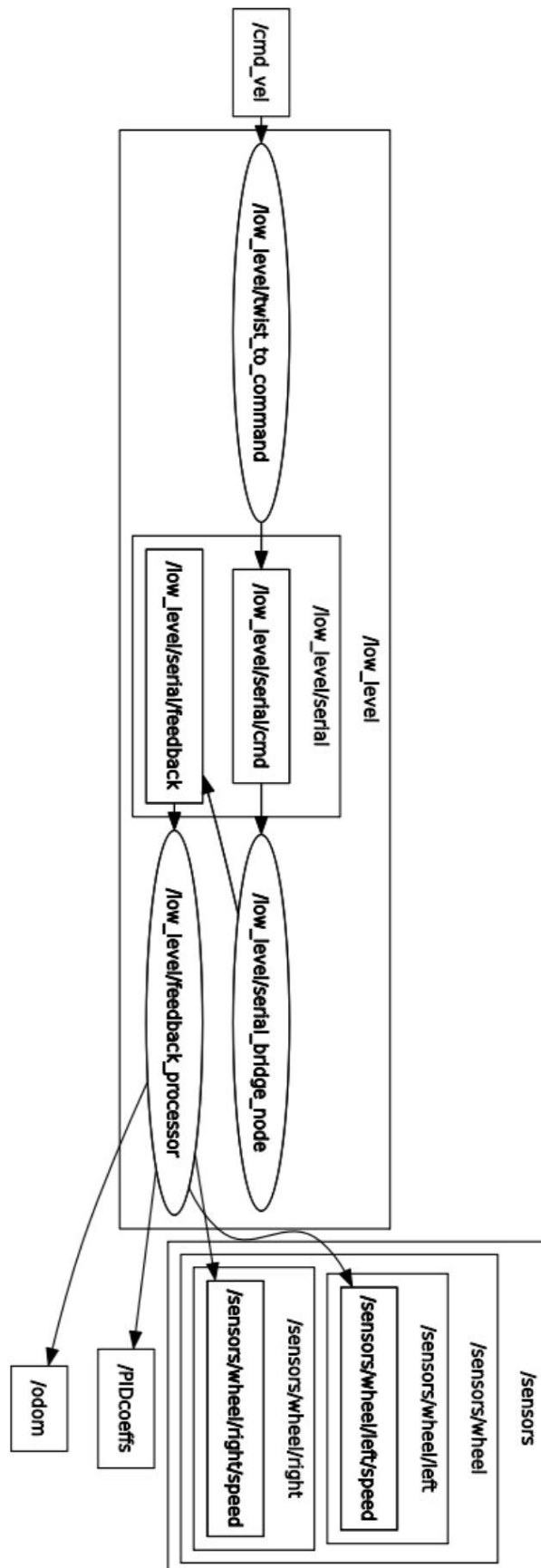


Рисунок 1 - RQT-граф, показывающий как взаимодействуют ноды, запускаемые Launch-файлом.

3. Особенности

Ноды из этого пакета работают с устройством, которое имеет название «/dev/esp32», что не является именем, которое даётся системой по умолчанию, а является симлинком (символической ссылкой). Это позволяет подключать ESP32 к разным физическим USB-портам, но в ОС они будут отображаться как одно устройство с фиксированным именем. Как создать такой симлинк описано в приложении В.

4. Развёртывание

Чтобы использовать функционал описанный выше, необходимо произвести подготовительные действия, описанные в приложении Г. В приложении представлен список действий по подготовке рабочего пространства и и установке зависимостей.

ROS2-пакет для настройки ПИД-регуляторов для моторов ESP32 - «simple_regulators_configurator_pkg»

1. Краткое описание

ROS2-нода, которая является python-приложением с графическим интерфейсом для настройки приводного уровня, а именно коэффициентов ПИД-регуляторов по скорости для каждого из моторов.

2. Возможности

Данная программа с графическим интерфейсом позволяет настроить коэффициенты ПИД-регуляторов и протестировать элементарное движение робота. В ПО реализованно:

- > **Управление роботом скоростями измеряемыми в СИ (м/с, рад/с).** Можно отправить сообщение-команду, в котором описан вектор скоростей базы (линейная скорость базы по оси X, угловая скорость базы вокруг оси Z) или вектор скоростей колёс (скорость левого колеса, скорость правого колеса).
- > **Просмотр графиков скоростей в реальном времени,** что помогает настроить регуляторы.
- > **Изменение коэффициентов ПИД-регуляторов** в удобном формате без необходимости перепрошивки ESP32.

- **Фиксация значений коэффициентов ПИД-регуляторов во Flash-памяти ESP32.**
- **Логгер отправляемых и получаемых сообщений.**
- **Кнопка экстренной остановки.**
- **Дистанционная настройка.** Физическая связь между ESP32 и компьютером, с которого происходит настройка не является обязательной.

Использование экосистемы ROS2 позволяет производить настройку дистанционно, что очень удобно в сравнении с проводным решением (см. следующий раздел).

3. Использование

Возможно 2 варианта использования:

- проводная настройка;
- беспроводная настройка.

Важный момент! Чтобы ПО работало корректно, необходимо на той машине, к которой Вы подключите ESP32 (зависит от типа подключения), произвести установку симлинка (подробнее в приложении В).

3.1. Проводная настройка

Все действия происходят на вашем компьютере. В нему подключаем ESP32 и там же запускаем GUI-приложение.

Для запуска системы необходимо:

1. Развернуть наш репозиторий (подробнее см. в приложении Г)
2. Открыть терминал и выполнить команду:

```
ros2 launch serial_bridge_package serial_bringup.launch.py
```

3. Открыть другой терминал и выполнить команду:

```
ros2 run simple_regulators_configurator_pkg simple_setup_node_gui
```

3.2. Беспроводная настройка

ESP32 подключено к Jetson Nano. Ваш компьютер (хост) подключён к той же сети, что и Jetson Nano. Подключаемся к Jetson Nano используя SSH (см. начало данного документа) и там запускаем набор нод для работы serial port. На хосте запускаем GUI-приложение.

3.2.1. Jetson Nano

Для запуска части системы необходимо:

1. Развернуть наш репозиторий (подробнее см. в приложении Г)

2. Открыть терминал и выполнить команду:

```
ros2 launch serial_bridge_package serial_bringup.launch.py
```

3.2.2. Ваш компьютер (хост)

Для запуска части системы необходимо:

3. Развернуть наш репозиторий (подробнее см. в приложении Г)

4. Открыть терминал и выполнить команду:

```
ros2 run simple_regulators_configurator_pkg simple_setup_node_gui
```

Python3-приложение для настройки ПИД-регуляторов для моторов ESP32 - «simple_PID_configurator»

1. Краткое описание

Python-приложение с графическим интерфейсом для настройки приводного уровня, а именно коэффициентов ПИД-регуляторов для каждого из моторов.

2. Возможности

Данная программа с графическим интерфейсом позволяет настроить коэффициенты ПИД-регуляторов и протестировать элементарное движение робота. В ПО реализованно:

- **Управление скоростями моторов командами использующими единицы измерения СИ (м/с, рад/с).** Можно отправить сообщение-команду, в которой описан вектор скоростей базы (линейная скорость базы по оси X, угловая скорость базы вокруг оси Z) или вектор скоростей колёс (скорость левого колеса, скорость правого колеса).
- **Просмотр графиков скоростей в реальном времени,** что помогает настроить регуляторы.
- **Изменение коэффициентов ПИД-регуляторов** в удобном формате без необходимости перепрошивки ESP32.
- **Запись значений коэффициентов ПИД-регуляторов в Flash-память ESP32.**
- **Логгер отправляемых и получаемых сообщений.**

➤ **Кнопка экстренной остановки.**

Параметры настройки находятся в файле «config.py». Там можно настроить скорость передачи и точку монтирования устройства.

3. Использование

Чтобы воспользоваться приложением, необходимо подключить ESP32 к компьютеру с монитором (это может быть как сам Jetson Nano, так и пользовательский компьютер).

Если используете симлинк, то его существование можно проверить (в выводе команды должно быть «/dev/esp32»):

```
ls /dev/esp32
```

Далее если ESP32 при подключении создаёт симлинк «/dev/esp32», то указать в файле-конфигурации это устройство. Если симлинк не создаётся, то рекомендуем это сделать (как сделать описывается в приложении В). Взаимодействие программы и ESP32 может происходить и без симлинка, но в таком случае ESP32 может в системе определяться по разному, т.е. в одном случае монтироваться как «/dev/ttyUSB0», а при переподключении - «/dev/ttyUSB1».

Чтобы запустить ПО, последовательно выполним следующие команды в терминале.

Установим pip и системную python-библиотеку:

```
sudo apt install python3-pip python3-tk
```

Установим зависимости:

```
pip install -r pip_requirements.txt
```

Запустим главную программу:

```
python3 fast_gui.py
```

ROS2-пакет для взаимодействия с лидаром -

«slidar_ros2»

1. Краткое описание

Это ROS2-пакет обеспечивающий взаимодействие с лидаром Slamtec RPlidar A2M8.

2. Возможности

Пакет позволяет получать информацию от лидара и публиковать её в топик (/scan) со стандартными для данной информации типами сообщениями.

3. Использование

1. Можно скачать отдельно

```
git clone https://github.com/Slamtec/sllidar_ros2.git
```

или использовать полный репозиторий (описано в приложении Г)

2. Необходимо создать симлинк для лидара. Сделать это можно по аналогии с тем, как создаётся симлинк для ESP32 (приложение В) или воспользоваться уже написанным скриптом.

```
cd sllidar_ros2  
sudo bash scripts/create_udev_rules.sh
```

3. Чтобы симлинк появился переподключите физически (выткните и воткните обратно USB-коннектор).

4. Чтобы запустить лидар, выполните команду

```
ros2 launch sllidar_ros2 sllidar_a2m8_launch.py
```

После запуска появится топик /scan с сообщениями типа sensor_msgs/msg/LaserScan и данными с лидара.

5. При остановке ноды, лидар может продолжить вращаться.

4. Ссылки

ROS2-пакет на Github: https://github.com/Slamtec/sllidar_ros2

ROS2-пакет для взаимодействия с камерой глубины

Взаимодействие с данным модулем ещё не было реализовано. Поэтому никаких материалов здесь нет. Но есть ROS2-пакет, который должен обеспечивать взаимодействие с ROS2-экосистемой. Данный тезис не проверялся практически. Ссылка представлена ниже.

<https://github.com/realsenseai/realsense-ros>

Оптическая камера

Взаимодействие с данным модулем ещё не было реализовано. Поэтому никаких материалов здесь нет.

Mini-RX MarvelMind

Взаимодействие с данным модулем ещё не было реализовано. Поэтому никаких материалов здесь нет. Но есть ROS2-пакет, который должен обеспечивать взаимодействие с ROS2-экосистемой. Данный тезис не проверялся практически. Ссылка представлена ниже.

https://github.com/MarvelmindRobotics/marvelmind_ros2_upstream

Ещё может быть полезным эта ссылка (типа готовое решение).

https://github.com/ipa-kut/ros_marvelmind_package

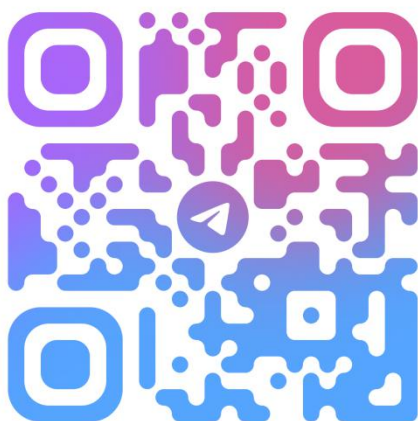
Полезные ссылки

- Репозиторий с ПО для Jetbot_mirea
https://github.com/cyberbanana777/mirea_jetbot_full
- Курс по ROS2
<https://stepik.org/course/221157/promo?search=8756811296>

Обратная связь

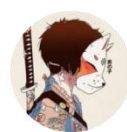
Если Вы нашли ошибку, неточность, у Вас есть предложения по улучшению или вопросы, напишите в Telegram Александру (https://t.me/Alex_19846) или Алисе (https://t.me/Kika_01). QR-коды со ссылками на их аккаунты представлены ниже.

Александр



@ALEX_19846

Алиса



@KIKA_01

ПРИЛОЖЕНИЕ А

Загрузка на ESP32 прошивки

Загрузку можно разбить на следующие этапы:

1. установка Arduino-IDE;
2. установка дополнительных библиотек;
3. загрузка прошивки;
4. проверка работы.

Рассмотрим каждый из них подробно.

1. Установка Arduino-IDE

На данный момент существует 2 глобальные версии Arduino-IDE: версии 1.x и 2.x. Мы будем использовать Arduino-IDE 2.x версии т.к. более простой способ установки библиотек.

Перейдём на раздел в Github-репозитория с релизами rduino-IDE <https://github.com/arduino/arduino-ide/releases>.

Среди доступных версий необходимо выбрать последнюю (версия 2.3.7 на момент написания - последняя доступная) и в разделе «Assets» скачать установщик под свою ОС, кликнув на необходимый файл. На рисунке 1 изображена интересующая нас область.

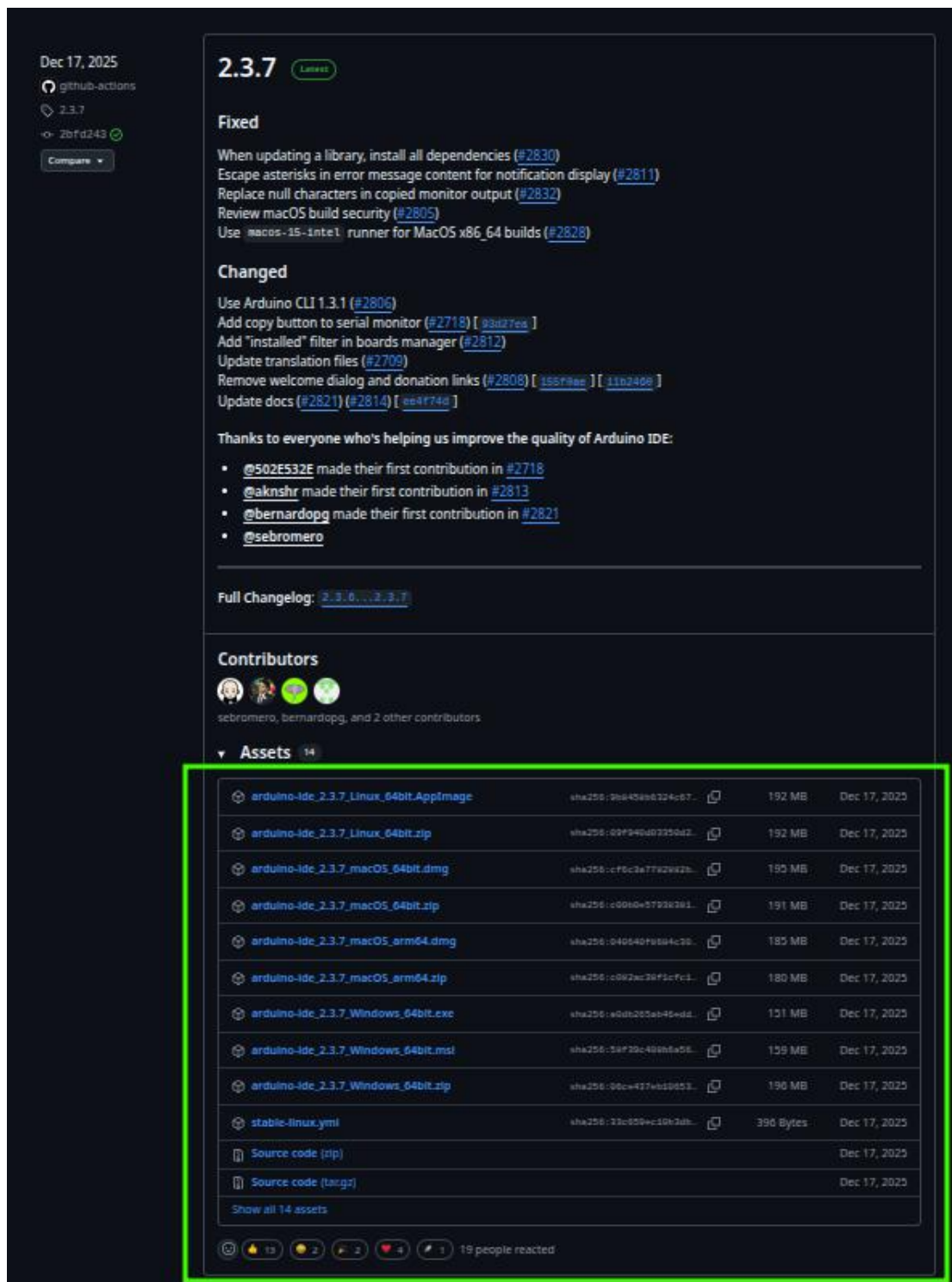


Рисунок 2 - Установщики Arduino-IDE 2.x.

Если у Вас Windows - загружаете и запускаете установщик (с расширением .exe) и следуете появляющимся инструкциям.

Если у Вас Linux - ниже инструкция для установки.

1. Скачиваете файл «arduino-ide_2.3.7_Linux_64bit.zip» в загрузки и выполняете команды:

```
cd ~/Downloads
unzip arduino-ide_2.3.7_Linux_64bit.zip
sudo cp -r ~/Downloads/arduino-ide_2.3.7_Linux_64bit /opt/
```

```
sudo ln -s /opt/arduino-ide_2.3.7_Linux_64bit/arduino-ide \
/usr/local/bin/arduino-ide
```

2. Настройка прав доступа к USB-портам (для работы с платами Arduino) Чтобы Arduino IDE могла работать с подключёнными платами, добавьте своего пользователя в группу «dialout»:

```
sudo usermod -a -G dialout $USER
```

3. Создаём ярлык (Nvim - текстовый редактор, Вы можете использовать любой редактор, который Вам удобен, например встроенный nano)

```
nvim ~/.local/share/applications/arduino-ide.desktop
```

И прописываем в файле такие настройки (чтобы появился ярлык в меню приложений):

```
[Desktop Entry]
Name=Arduino IDE
Comment=Arduino Integrated Development Environment
Exec=/usr/local/bin/arduino-ide
Icon=/opt/arduino-
ide_2.3.7_Linux_64bit/resources/app/resources/icons/512x512.png
Terminal=false
Type=Application
Categories=Development;IDE;
```

4. Делаем ярлык исполняемым:

```
chmod +x ~/.local/share/applications/arduino-ide.desktop
```

5. Обновляем кэш

```
update-desktop-database ~/.local/share/applications/
```

6. Можно удалить исходники:

```
sudo rm -r ~/Downloads/arduino-ide_2.3.7_Linux_64bit
sudo rm -r ~/Downloads/arduino-ide_2.3.7_Linux_64bit.zip
```

После этих действий Вы можете запустить arduino-ide. Достаточно в поиске меню приложений вбить «arduino» и кликнуть на пиктограмму приложения.

В результате будет показано окно, отображённое на рисунке 3.

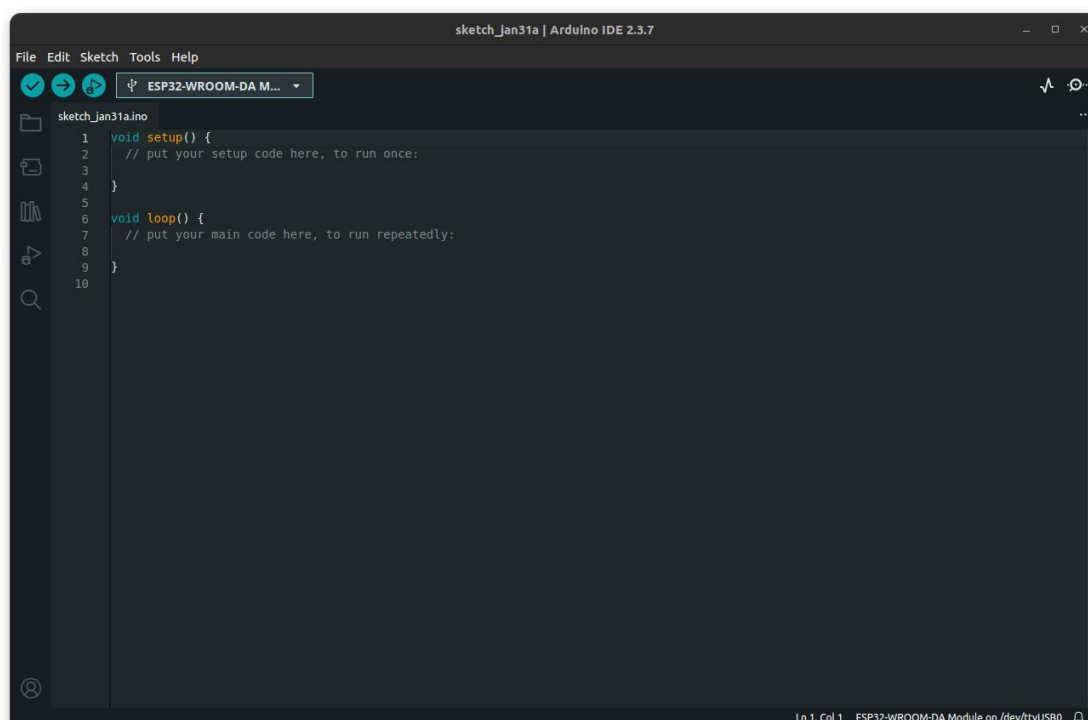


Рисунок 3 - Главное окно Arduino-IDE.

2. Установка дополнительных библиотек

Далее необходимо установить библиотеки, которые используются в прошивке и пакеты для работы с платой ESP32.

Нам понадобятся библиотеки:

- GyverPID (на момент написания последняя доступная версия 3.3.2)(подробнее про функционал библиотеки можно прочитать здесь <https://github.com/GyverLibs/GyverPID>);
- Adafruit PWM Servo Driver Library (на момент написания последняя доступная версия 3.0.2)(подробнее про функционал библиотеки можно прочитать здесь <https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library>).

Установка производится простым способом: нужно выбрать вкладку слева «Менеджер библиотек» или «Library Manager» и ввести название библиотеки. Далее выбрать версию и нажать на кнопку «Установить» или «Install». Процесс показан на рисунке 4.

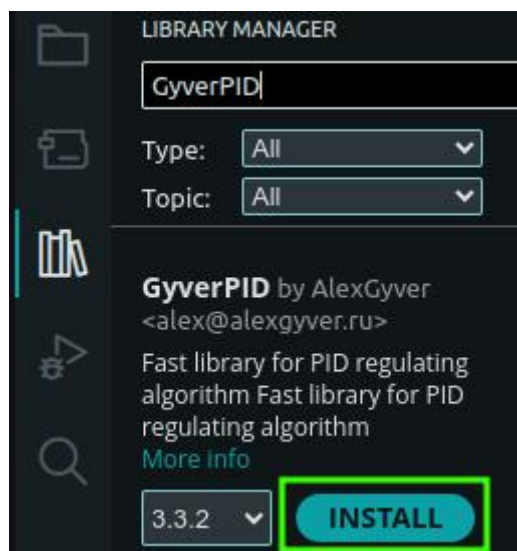


Рисунок 4 - Пример установки библиотеки.

Во время установки библиотеки «Adafruit PWM Servo Driver Library» откроется окно, в котором предлагают установить зависимости данной библиотеки. Выбираем «Установить всё» или «Install all». Процесс показан на рисунке 5.

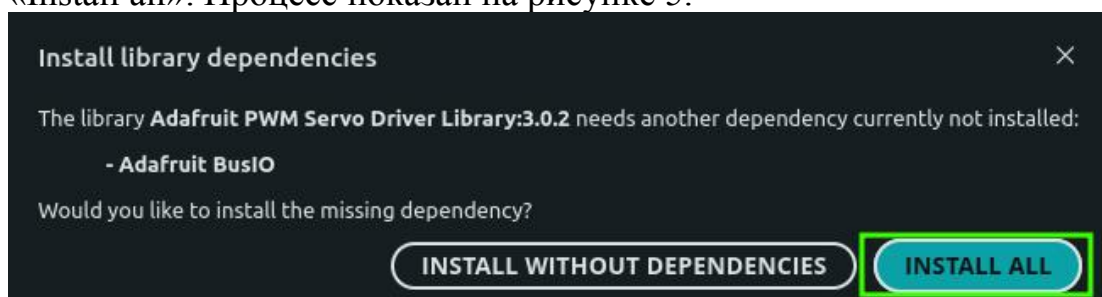


Рисунок 5 - Установка зависимостей библиотеки.

Далее необходимо, чтобы установить пакет для работы с платой ESP32, исправить файл «arduino-cli.yaml».

В Windows этот файл находится по адресу:

`C:\Users\<ваше_имя_пользователя>\.arduinoIDE\arduino-cli.yaml`

в Linux:

`/home/<ваше_имя_пользователя>/.arduinoIDE/arduino-cli.yaml`

В этом файле необходимо дописать (с помощью `nvim` или Вашего любимого текстового редактора) в конец следующие строки

```
network:
  connection_timeout: 600s
```

чтобы содержимое файла выглядело так, как показано на рисунке 6.

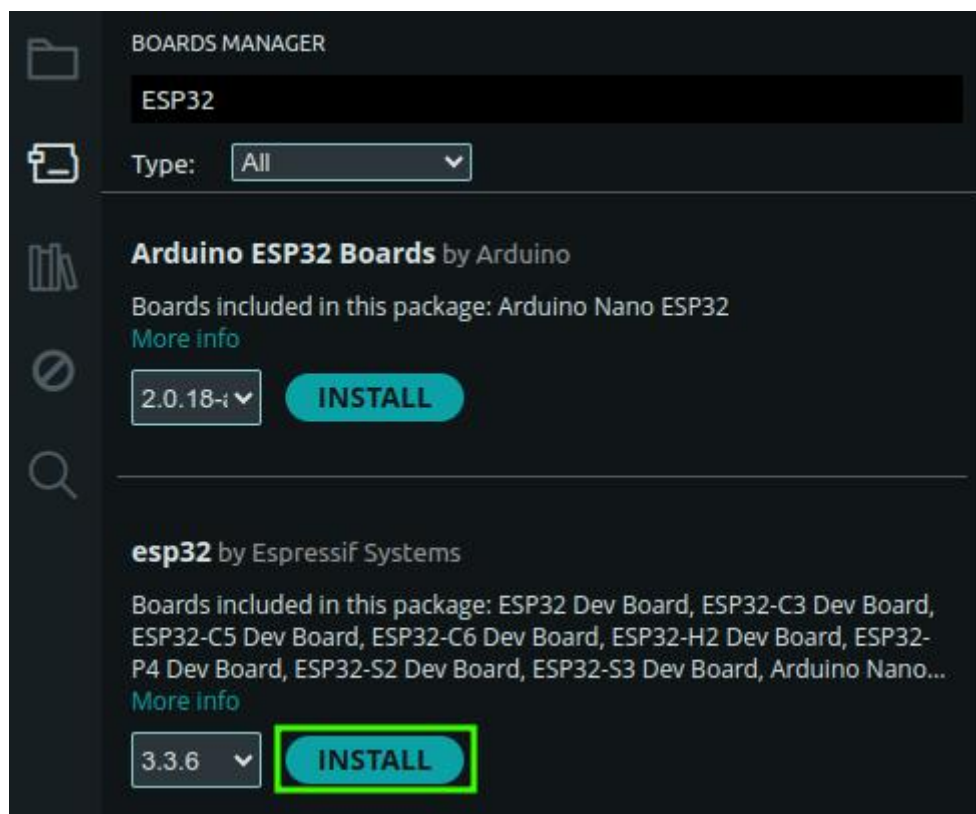


Рисунок 7 - Установка пакета для работы с платами ESP32.

3. Прошивка ESP32

Подключаем EPS32 к компьютеру физически (через USB) и отрываем скетч прошивки, расположенный в репозитории ПО для Jetbot (https://github.com/cyberbanana777/mirea_jetbot_full) по адресу <путь к репозиторию>/esp_firmware/esp32_firmware/esp32_firmware.ino

Далее необходимо нажать на кнопку выбора платы (изображена на рисунке 8).

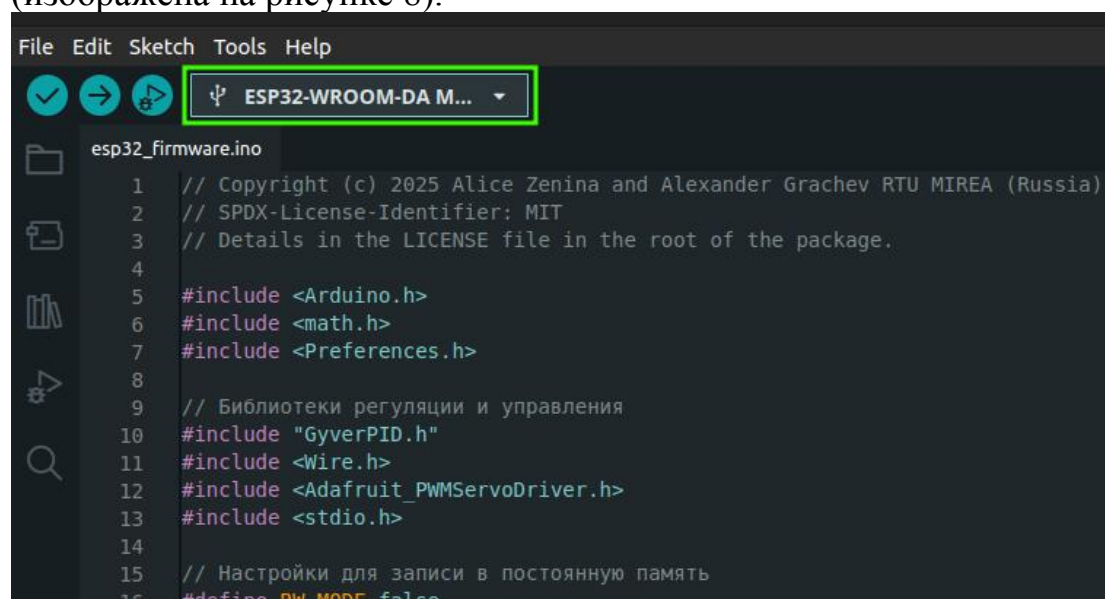


Рисунок 8 - Кнопка выбора платы.

При нажатии на неё выпадет список и в нём необходимо нажать на кнопку «Выбрать другую плату и порт» или «Select other board and port» (изображена на рисунке 8).

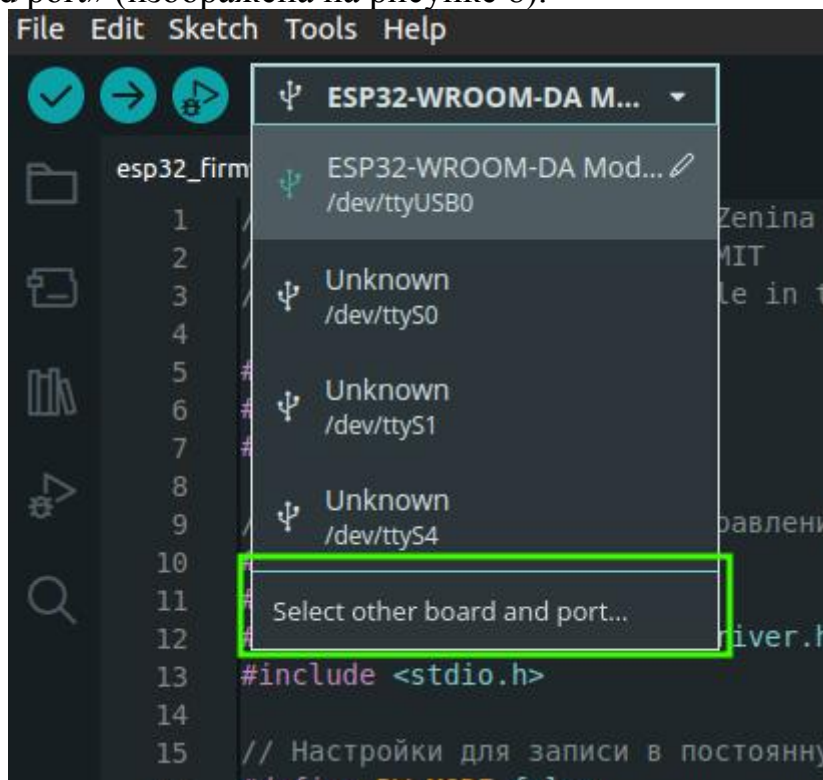


Рисунок 9 - Кнопка настройки платы и порта.

После нажатия откроется окно в левой части которого нужно выбрать плату, а в правой - порт. Для платы мы используем название «ESP32-WROOM-DA Module», а порт выбираем «/dev/ttyUSB0». Названия для портов в Windows отличаются от представленного примера (на примере изображена Linux-версия). Также если Вы не можете однозначно определить порт (к примеру у Вас подключено несколько устройств и отражаются они как «/dev/ttyUSB0», «/dev/ttyUSB1» и т.д., то рекомендуем отключить все другие устройства кроме ESP32 на момент прошивки). Процесс настройки платы и порта показан на рисунке 10.

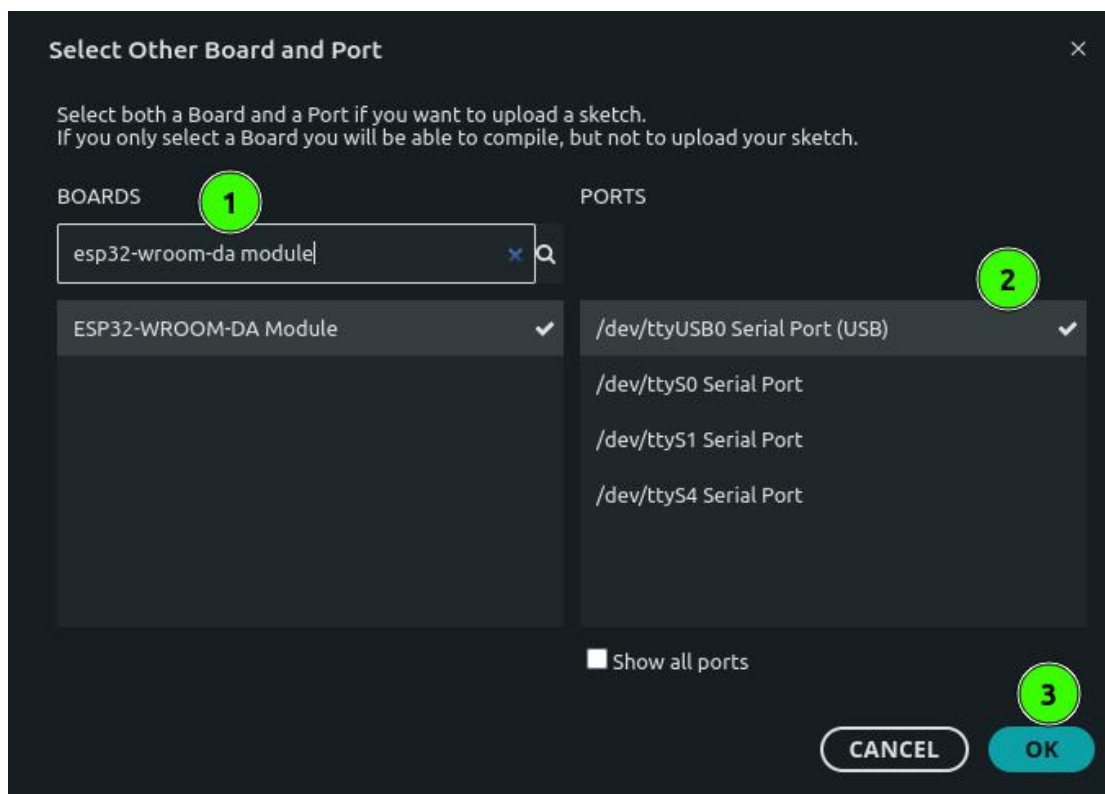


Рисунок 10 - Процесс настройки платы и порта.

Далее рассмотрим дополнительную настройку - стирание Flash-памяти при перепрошивке. Если Вы ходите сбросить значения коэффициентов до значения по умолчанию при перезаливке кода, то выберете значение «Enable» или «Включить», если Вы хотите отставить значения коэффициентов, сохранённые в памяти, то выберете значение «Disable» или «Выключить». Расположение данной опции представлено на рисунке 11.

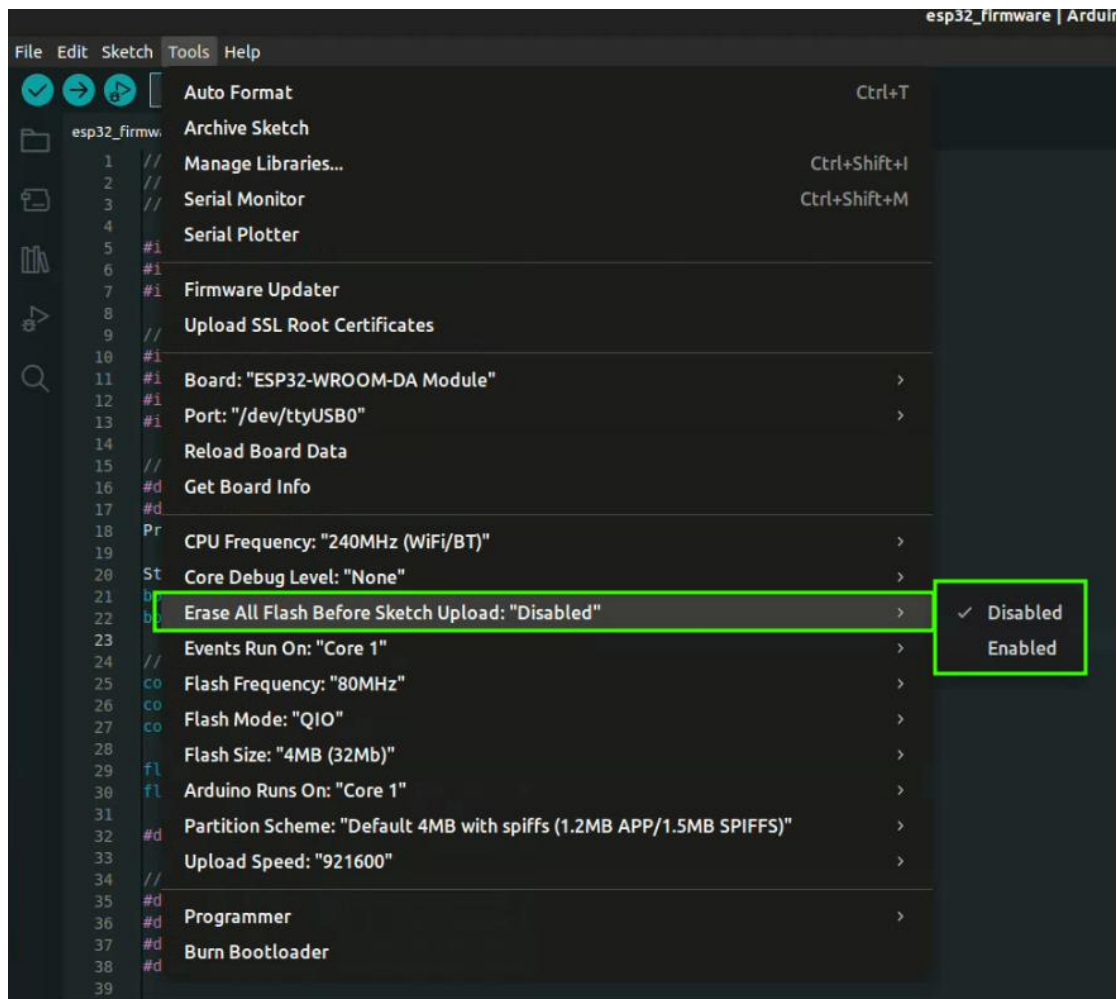


Рисунок 11 - Опция очистки Flash-памяти

Далее необходимо запустить процесс обновления ПО - необходимо нажать на кнопку загрузки, изображённой на рисунке 12.

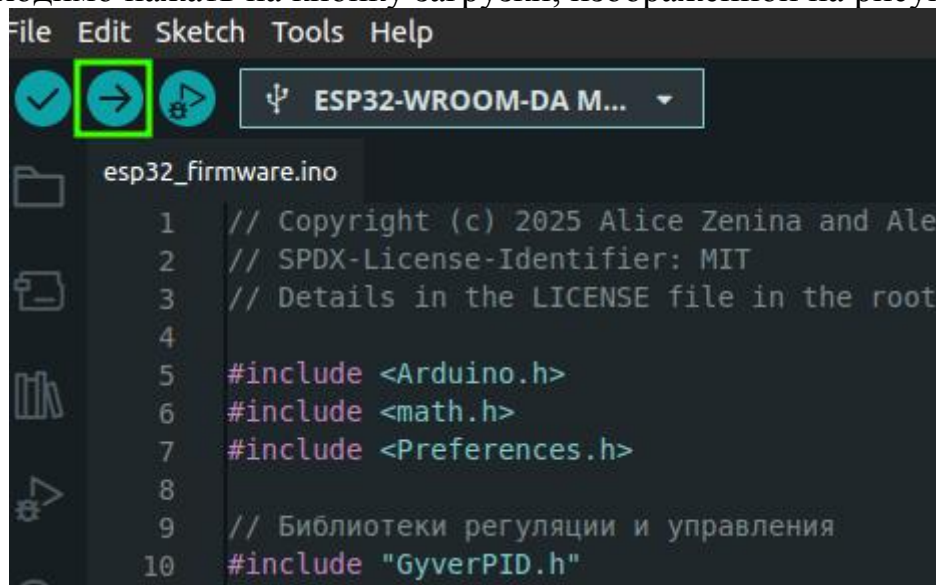


Рисунок 12 - Кнопка загрузки прошивки.

После этого некоторого времени ожидания, во вкладке «Вывод» или «Output» мы увидим примерно такую картину,

изображённую на рисунке 13, что свидетельствует о корректной загрузке прошивки ESP32.

```
Output
Writing at 0x00001000 [ ] 0.0% 0/16076 bytes...

Writing at 0x00007260 [=====] 100.0% 16076/16076 bytes...
Wrote 25184 bytes (16076 compressed) at 0x00001000 in 0.4 seconds (546.4 kbit/s).
Hash of data verified.
Compressed 3072 bytes to 146...

Writing at 0x00008000 [ ] 0.0% 0/146 bytes...

Writing at 0x00008c00 [=====] 100.0% 146/146 bytes...
Wrote 3072 bytes (146 compressed) at 0x00008000 in 0.0 seconds (1589.9 kbit/s).
Hash of data verified.
Compressed 8192 bytes to 47...

Writing at 0x0000e000 [ ] 0.0% 0/47 bytes...

Writing at 0x00010000 [=====] 100.0% 47/47 bytes...
Wrote 8192 bytes (47 compressed) at 0x0000e000 in 0.0 seconds (2188.0 kbit/s).
Hash of data verified.
Compressed 342080 bytes to 193186...

Writing at 0x00010000 [ ] 0.0% 0/193186 bytes...

Writing at 0x0001c209 [=> ] 8.5% 16384/193186 bytes...

Writing at 0x0002a71d [====> ] 17.0% 32768/193186 bytes...

Writing at 0x0002fcee [=====> ] 25.4% 49152/193186 bytes...

Writing at 0x000357de [=====> ] 33.9% 65536/193186 bytes...

Writing at 0x0003aec4 [=====> ] 42.4% 81920/193186 bytes...

Writing at 0x00040201 [=====> ] 50.9% 98304/193186 bytes...

Writing at 0x00045a87 [=====> ] 59.4% 114688/193186 bytes...

Writing at 0x0004b02f [=====> ] 67.8% 131072/193186 bytes...

Writing at 0x0005097c [=====> ] 76.3% 147456/193186 bytes...

Writing at 0x00059122 [=====> ] 84.8% 163840/193186 bytes...

Writing at 0x0005ed03 [=====> ] 93.3% 180224/193186 bytes...

Writing at 0x00063840 [=====] 100.0% 193186/193186 bytes...
Wrote 342080 bytes (193186 compressed) at 0x00010000 in 3.0 seconds (919.2 kbit/s).
Hash of data verified.

Hard resetting via RTS pin...
```

Рисунок 13 - Консольный вывод, свидетельствующий от нормальной загрузке прошивки ESP32.

Далее необходимо перезапустить ESP32. Это можно сделать переподключив её к USB-коннектору или нажав на кнопку «EN», расположенную на плате.

4. Проверка работы ПО

5. Чтобы проверить корректность работы прошивки и ESP32 - нажмите на кнопку открытия Serial Port'а, изображённую на рисунке 14.

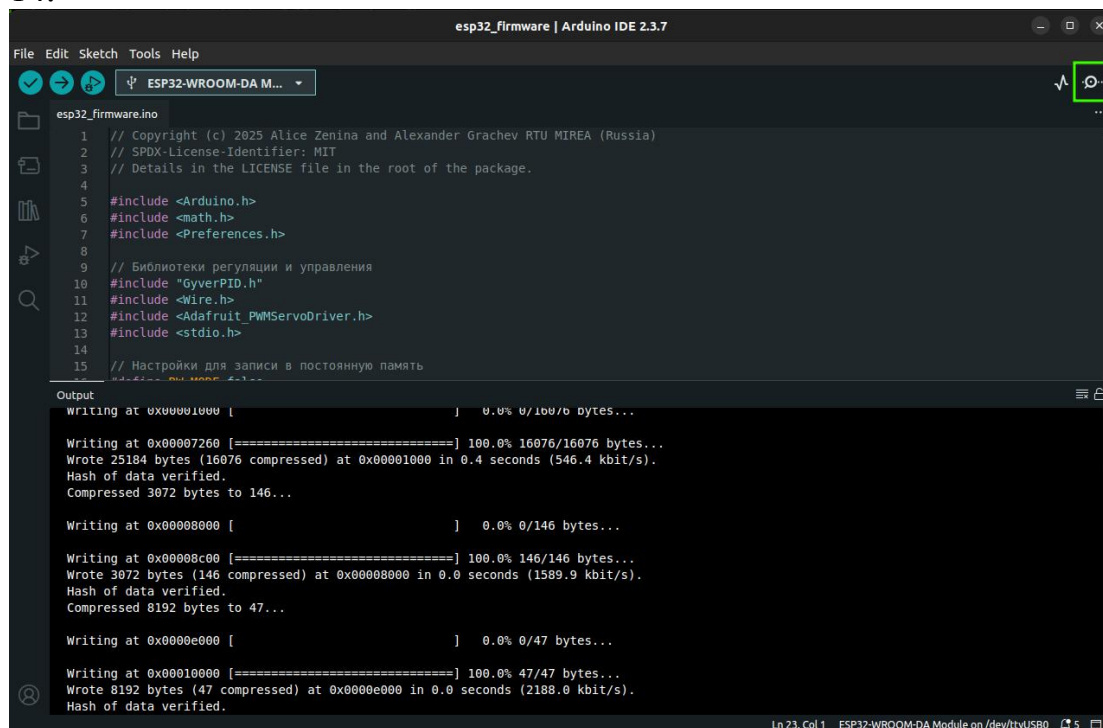


Рисунок 14 - Кнопка открытия Serial Port.

После нажатия откроется вкладка «Serial Monitor». Если вывод будет невнятным, то необходимо справа выбрать правильную скорость Serial Port - используем 115200 baud. Процесс изображён на рисунке 15.

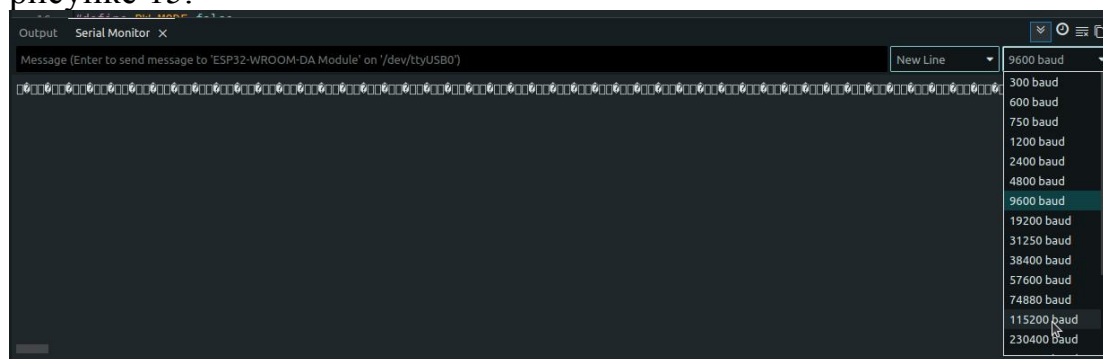


Рисунок 15 - Настройка монитора порта.

После настройки монитора порта вывод будет примерно таким (цифры могут отличаться, но структура будет такой же), как изображено на рисунке 16.

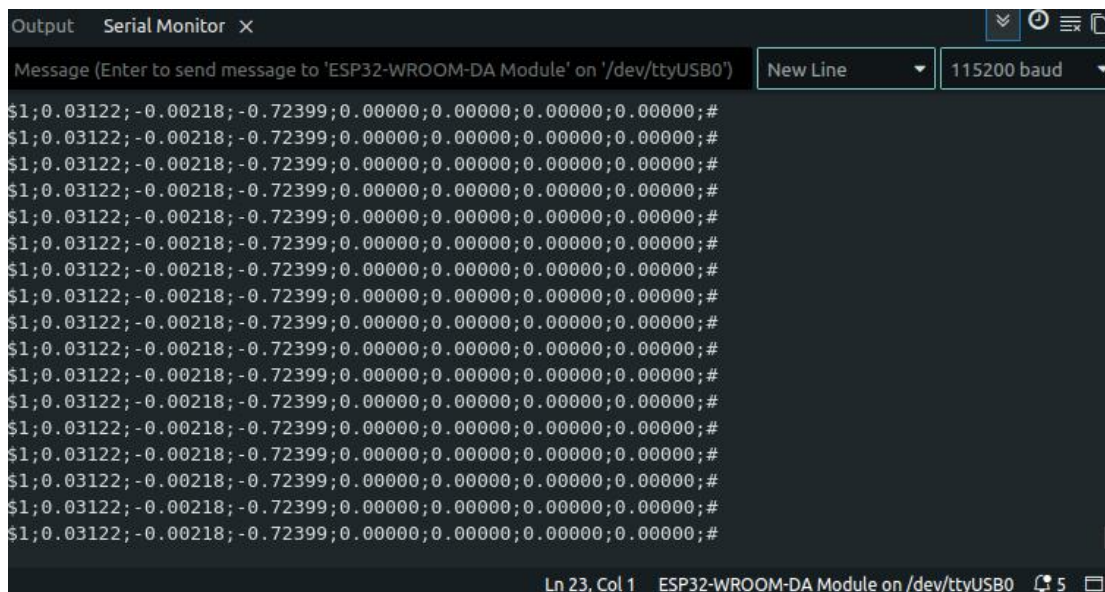


Рисунок 16 - Нормальный вывод Serial Port.

Это сообщения-ответы типа 1 (1 тип сообщений кастомного протокола). Подробнее Вы можете ознакомиться с протоколом передачи данных в приложении Б.

ПРИЛОЖЕНИЕ Б

Протокол взаимодействия с ESP32

ESP32 можно рассматривать как конечный автомат, который принимает сообщения-команды и отправляет сообщения-ответы. На рисунке 17 представлены все реализованные команды в виде схемы.

Сообщения-команды (входящие)

1. Управление скоростями платформы.

Вид:

\$1;V_linear_x;V_angular_z;#

Поля:

- V_linear_x - желаемая линейная скорость платформы по оси X в м/с;
- V_angular_z - желаемая угловая скорость платформы вокруг оси Z в рад/с.

2. Управление скоростями каждого из колёс.

Вид:

\$2;left_wheel_speed;right_wheel_speed;#

Поля:

- left_wheel_speed - желаемая скорость левого колеса в м/с;
- right_wheel_speed - желаемая скорость правого колеса в м/с.

3. Установка коэффициентов ПИД-регуляторов для управления скоростью моторов.

Вид:

\$3;Kp_L;Ki_L;Kd_L;Kp_R;Ki_R;Kd_R;#

Поля:

- Kp_L - пропорциональный коэффициент регулятора левого мотора;
- Ki_L - интегральный коэффициент регулятора левого мотора;
- Kd_L - дифференциальный коэффициент регулятора левого мотора;
- Kp_R - пропорциональный коэффициент регулятора правого мотора;
- Ki_R - интегральный коэффициент регулятора правого мотора;

- Kd_R - дифференциальный коэффициент регулятора правого мотора.

4. Запись коэффициентов ПИД-регуляторов для управления скоростью моторов во Flash-память (энергонезависимую).

Вид:

\$4;Kp_L;Ki_L;Kd_L;Kp_R;Ki_R;Kd_R;#

Поля:

- Kp_L - пропорциональный коэффициент регулятора левого мотора;
- Ki_L - интегральный коэффициент регулятора левого мотора;
- Kd_L - дифференциальный коэффициент регулятора левого мотора;
- Kp_R - пропорциональный коэффициент регулятора правого мотора;
- Ki_R - интегральный коэффициент регулятора правого мотора;
- Kd_R - дифференциальный коэффициент регулятора правого мотора.

5. Запрос текущих коэффициентов ПИД-регуляторов.

Вид:

\$5;#

Поля: -

Сообщения-ответы (исходящие)

1. Получение одометрии и скоростей платформы и колёс.

Вид:

\$1;odom_x;odom_y;odom_theta;V_linear_x;V_angular_z;left_wheel_speed;right_wheel_speed;#

Поля:

- odom_x - координата по оси x положения на плоскости робота в м;
- odom_y - координата по оси y положения на плоскости робота в м;
- odom_theta - угол, на который развёрнут робот, относительно положения запуска на плоскости в рад;

- V_linear_x - текущая линейная скорость платформы по оси X в м/с;
- $V_angular_z$ - текущая угловая скорость платформы вокруг оси Z в рад/с;
- $left_wheel_speed$ - текущая скорость левого колеса в м/с;
- $right_wheel_speed$ - текущая скорость правого колеса в м/с.

2. Получение текущих коэффициентов ПИД-регуляторов для управления скоростью моторов. (ответ на сообщение-команду 5)

Вид:

\$2;Kp_L;Ki_L;Kd_L;Kp_R;Ki_R;Kd_R;#

Поля:

- Kp_L - пропорциональный коэффициент регулятора левого мотора;
- Ki_L - интегральный коэффициент регулятора левого мотора;
- Kd_L - дифференциальный коэффициент регулятора левого мотора;
- Kp_R - пропорциональный коэффициент регулятора правого мотора;
- Ki_R - интегральный коэффициент регулятора правого мотора;
- Kd_R - дифференциальный коэффициент регулятора правого мотора.

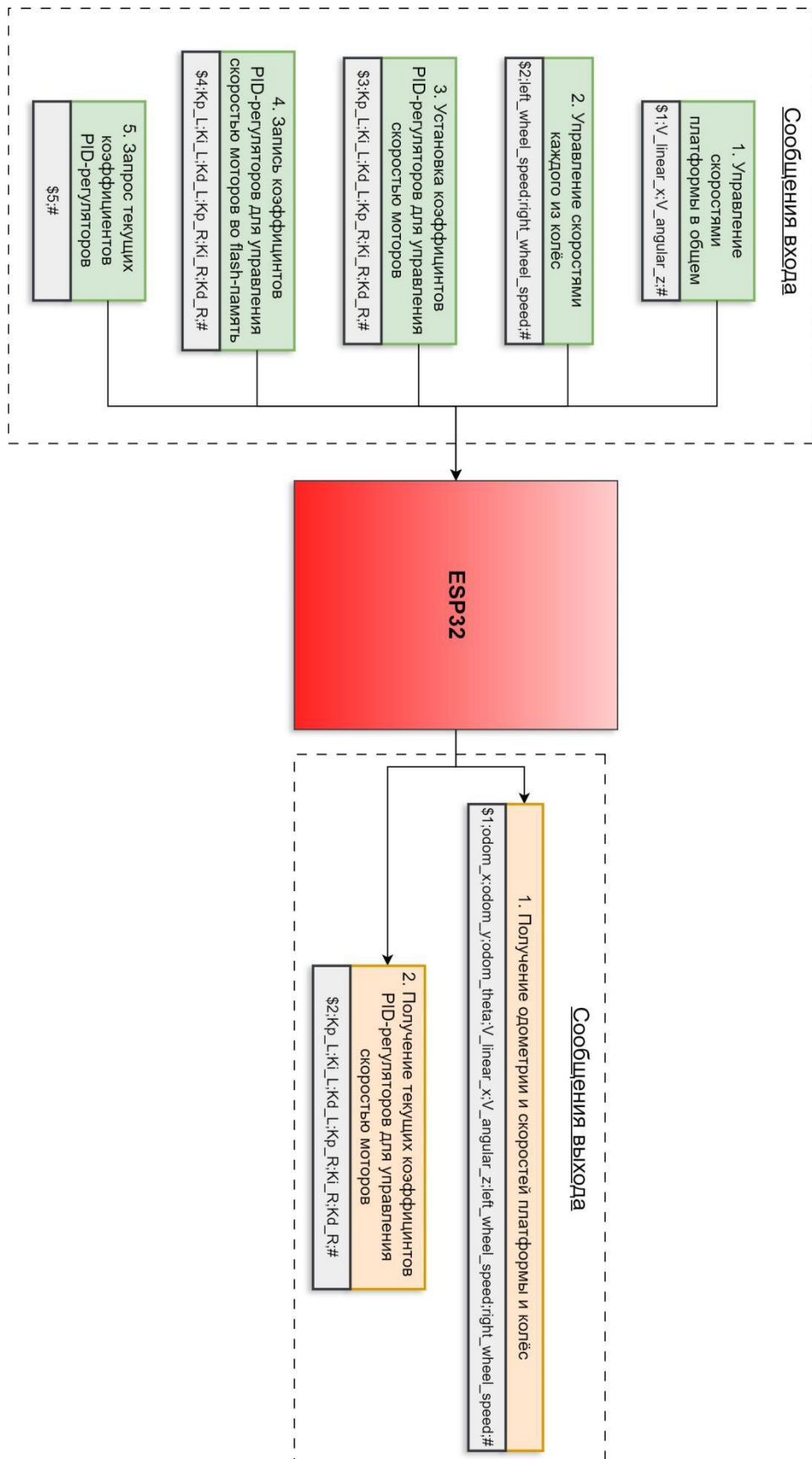


Рисунок 17 - Сообщения для взаимодействия с ESP32.

ПРИЛОЖЕНИЕ В

Создание симлинка для ESP32

1. Перейдите в папку

```
cd <путь к репозиторию>/init_scripts/esp32/
```

2. В терминале выполните (без подключенного ESP32):

```
lsusb > without_device.txt
```

3. Подключите устройство (ESP32) к компьютеру

4. В терминале выполните:

```
lsusb > with_device.txt
```

5. Сравните два файла и найдите разницу с помощью этого скрипта:

```
sudo bash add_symlink_for_esp32.bash
```

6. Удалите временные файлы:

```
rm without_device.txt  
rm with_device.txt
```

7. Физически переподключите устройство (ESP32) к компьютеру

8. Проверьте, что оно успешно определилось системой:

```
ls /dev/esp32
```

Немного о том, что происходит под капотом этого скрипта

Утилита «lsusb» позволяет получить информацию о подключённых устройствах через USB. Каждое устройство (разные) имеет уникальные ProductId и VendorId, позволяющие единственным образом идентифицировать в системе (одинаковые ESP32, как на jetbot'ах, имеют одинаковые ProductId и VendorId). Мы фиксируем информацию о подключённых устройствах без ESP32 в файл «without_device.txt», затем подключаем ESP32 и вновь фиксируем информацию, но уже в другой файл «with_device.txt». Затем программно сравниваем эти файлы и определяем ProductId и VendorId. Далее создаём правило для службы администрирования физических подключений, используя в правиле ProductId и VendorId. Правило задаёт устройству постоянное имя, независимо от того, к какому физическому порту оно подключено. Все скрипты и сервисы могут обращаться к устройству по этому имени.

Затем перезапускаем службу с учётом правил. После всего, необходимо физически переподключить устройство (ESP32), чтобы система применила правило.

ПРИЛОЖЕНИЕ Г

Развёртывание репозитория с ПО для jetbot_mirea

В терминале необходимо последовательно выполнить команды:

1. Создаём рабочее пространство и переходим в папку исходников.

```
mkdir -p jetbot_ws/src  
cd jetbot_ws/src
```

2. Скопируем репозиторий.

```
git clone  
https://github.com/cyberbanana777/mirea_jetbot_full.git .
```

3. Установим зависимости.

```
sudo bash init_scripts/install_deps.bash
```

4. Произведём сборку рабочего пространства.

```
cd ../../  
colcon build  
source install/local_setup.bash
```

5. Добавим в авто-source при каждом запуске терминала.

```
line_to_add="source \"$(pwd)/install/local_setup.bash\""  
grep -qxF "$line_to_add" ~/.bashrc || echo "$line_to_add" >>  
~/.bashrc
```